

Data Handling, Statistics & DoE

for Materials Scientists

Lecture 1 · 2 × 45 min

Course introduction · Working with data & collaboration · Part II: Data Handling & Materials Ontologies

1KB570 · 5 credits · Second cycle

Department of Chemistry – Ångström, Uppsala University

Today's Agenda

2 x 45 min, roughly in this order:

1. Why data, why now — the big picture
2. Course layout: lectures, live tutorials, laborations, project
3. Prerequisites, extra material & the course book
4. Using LLMs well in this course
5. Working with data in this course
6. Collaboration: Git & GitHub
7. *Break*
8. **Part II: Data Handling & Materials Ontologies**
9. Preparing for the two live-tutorial workshops
10. Wrap-up & what's next

Course Layout: How This Course Runs

Component	What it is
Lectures	Concept introductions, like this one — one per major Part
Live tutorials	10 hands-on sessions across the book, worked live at the keyboard
Laborations	3 lab exercises (next slide)
Course project	An independent analysis of your choosing, presented not written up

Examination: a written exam, the mandatory laborations, and the course project together — not one single grade for the whole course.

Everything above is built around one Jupyter Book — **your course book**, covered in full a few slides from now.

Three Laborations

#	Topic	Status
1	Part I refresher — make sure your Python environment works, ask questions if you feel rusty	<i>Optional</i>
2	Wine Quality: data curation → PCA → PLS or LDA	Mandatory
3	DoE Optimisation Contest — find a process optimum in as few experiments as possible, against a live "plant" server	Mandatory

Laborations 2 and 3 are both **4-hour sessions with checkpoint questions instead of a written report** — you demonstrate understanding by answering directly, not by writing up a lab report afterwards.

Laboration 1 is not graded, but if Part I (Python basics) feels shaky, that session is exactly the place to fix it — with help in the room — before it becomes a problem in Laboration 2.

The Course Project

Pick a **publicly available materials-science dataset**, apply what you've learned across the course, and **present your findings in a 10-minute talk** — that presentation, not a written report, is the graded deliverable. A reproducible Jupyter Notebook backs the talk, but you don't submit it as a stand-alone document.

- Choose your own dataset (suggestions provided) or one of five ready-made project topics
- At least four of: descriptive stats, a hypothesis test, a regression/PLS model, PCA/LDA, a DoE-style optimisation, or a method comparison
- Optional: compare a classical method against one from Part VI (AI/ML)

Full details, suggested datasets, and grading criteria: Part VII of the course book.

Prerequisites & Extra Material

Prerequisites

- Basic Python — Part I of the book is a self-study refresher, not a formal requirement you're assumed to have passed elsewhere
- No prior statistics or DoE background assumed — this course builds both from the ground up

Extra material

- **Part VI: Intro to AI & Machine Learning** is explicitly *optional*, beyond the core syllabus — clustering, Gaussian processes, Bayesian optimisation, ensembles, neural networks
- Useful for the course project's optional ML-comparison step, and worth exploring if you want to go further than the graded material requires

This Jupyter Book Is Your Textbook

Everything in this course — theory, worked examples, live tutorials, labs, the project guide — lives in one place:

`peterbmob.github.io/1KB570`

- Every single notebook page has an **Open in Colab** button — click it, and you're editing and running that exact page's code in your browser, no install, no account needed
- This lecture itself is one of those notebooks — the live-runnable slides you're looking at right now

The Book's Structure: Seven Parts

Part	Topic	Key tools
I	Prerequisites — Python for Data Analysis	numpy, pandas, matplotlib, plotly
II	Data Handling & Materials Ontologies ← today	json, pandas, owlready2, rdflib
III	Applied Statistics	scipy.stats, statsmodels
IV	Multivariate Analysis	scikit-learn, factor_analyzer
V	Design of Experiments	pyDOE3, scipy.optimize
VI	Intro to AI & Machine Learning (extra)	scikit-learn
VII	Course Project	everything above

Why Data, Why Now

- **Science is producing more data than any one person can manually curate**
 - Instruments, simulations, and high-throughput experiments now generate data faster than it can be read, let alone re-checked, by hand
 - The bottleneck is shifting from "*how do I get a measurement*" to "*how do I find, trust, and reuse the measurements that already exist*"
- **Research is becoming more collaborative and more interdisciplinary**
 - A single materials project now routinely spans synthesis chemists, computational scientists, and data/ML specialists — often across institutions and countries
 - That only works if data means the same thing to everyone touching it — exactly the problem Part II's FAIR principles and ontologies (EMMO) solve
- **Modern AI and machine learning are fundamentally data-hungry**
 - A model is only as good as the data it was trained and validated on
- **Reproducibility has become a serious concern across science**
 - Results that can't be reproduced — because data or methods were never properly documented — are a real, costly problem

Materials Science Is Part of This Shift

- From single-sample, single-lab studies toward **shared, data-driven infrastructure**
 - High-throughput experimentation, autonomous/self-driving labs, digital twins
 - Large public materials databases now hold millions of entries — **NOMAD**, Materials Project, AFLOW, OQMD
- These infrastructures only work because of the ideas in **Part II, today**
 - FAIR metadata, shared ontologies (EMMO), structured queryable databases
- Funders increasingly **require** this — EU Horizon Europe and most national research councils now expect a data management plan and FAIR data as standard practice
- None of this works without people from different backgrounds agreeing on what a field even *means* — the interdisciplinary/collaboration point from the previous slide, made concrete

*Today's second workshop puts you inside a real example of this shift: querying **NOMAD**, a live, public, production-scale materials-data infrastructure, yourself.*

The Three Pillars of This Course

Everything in this book fits under three questions, in this order:

Pillar	Question	Part
1. Handle	Is your data structured and described well enough to trust and reuse?	Part II ← today
2. Analyze	Given data you already have, what can you honestly conclude from it?	Parts III–IV
3. Generate	If you need <i>more</i> or <i>better</i> data, how do you get the most out of every expensive experiment?	Part V (DoE)

Handling data badly wastes the analysis; analysing badly-designed data wastes the experiment. **DoE (Part V) closes the loop** — it's literally the discipline of generating data efficiently in the first place, which is why it's the course's final methodological Part before the project.

Using LLMs Well in This Course

You are allowed to use LLMs (ChatGPT, Claude, Copilot, ...) in this course. With every notebook now one click away in Colab, there's nothing stopping you from pasting code in and running it — so this needs saying plainly:

- **You are examined on understanding, not on producing working code.** The written exam tests whether *you* can choose the right test, read a p-value, design a factorial, interpret a loading plot — with no LLM in the room.
- **Using an LLM to skip that understanding now is a loan against the exam later.** Code that runs today because an LLM wrote it doesn't teach you anything the exam will ask you to reproduce.
- **Using an LLM to build understanding is genuinely useful** — and encouraged.

The difference isn't whether you use one — it's what you ask it for.

Good Prompt vs. Bad Prompt

✗ Fast-completion (avoid)	✓ Hint-seeking (encouraged)
<i>"Write the code to run a two-sample t-test on this data"</i>	<i>"I think I need a two-sample t-test here — what should I check before trusting one?"</i>
<i>"Fix this error for me" (paste, get code back, move on)</i>	<i>"Explain what this error message means, in plain terms"</i>
<i>"What's the answer to Checkpoint 2b?"</i>	<i>"Here's my reasoning for Checkpoint 2b — does it hold together?"</i>
Copy → paste → run → hand in	Read the explanation → try it yourself → use the LLM to check your reasoning

The left column gets you a working cell today and a blank page in the exam hall. The right column is slower right now and is exactly what actually prepares you for the written exam — treat an LLM like a very patient teaching assistant who won't just do the exercise for you.

Working with Data in This Course

Why almost every notebook you will see — today and for the rest of the course — starts from simulated data with a known right answer

Why Generated (Simulated) Data?

Every case study in this course — ZnO grain size, NMC cathode capacity, LiFePO₄ batches, TiO₂ photocatalysis — starts from a **known true relationship plus random noise**. That is deliberate:

1. **You can check whether a method actually recovers the truth** — a t-test, a PCA, a factorial design — because you built the true answer in yourself. Real data never lets you do that.
2. **It's fully reproducible** — same seed, same numbers, same discussion, for every student, every time
3. **It's safe and fast** — no lab time, no reagent cost; try 10,000 experiments before running one real one
4. **It mirrors real practice** — this is exactly what a statistician calls a *Monte Carlo check*

A Synthetic Dataset, Live

Simulating 100 charge-discharge cycles of a battery — exponential capacity fade plus measurement noise. **Run the cell below live.**

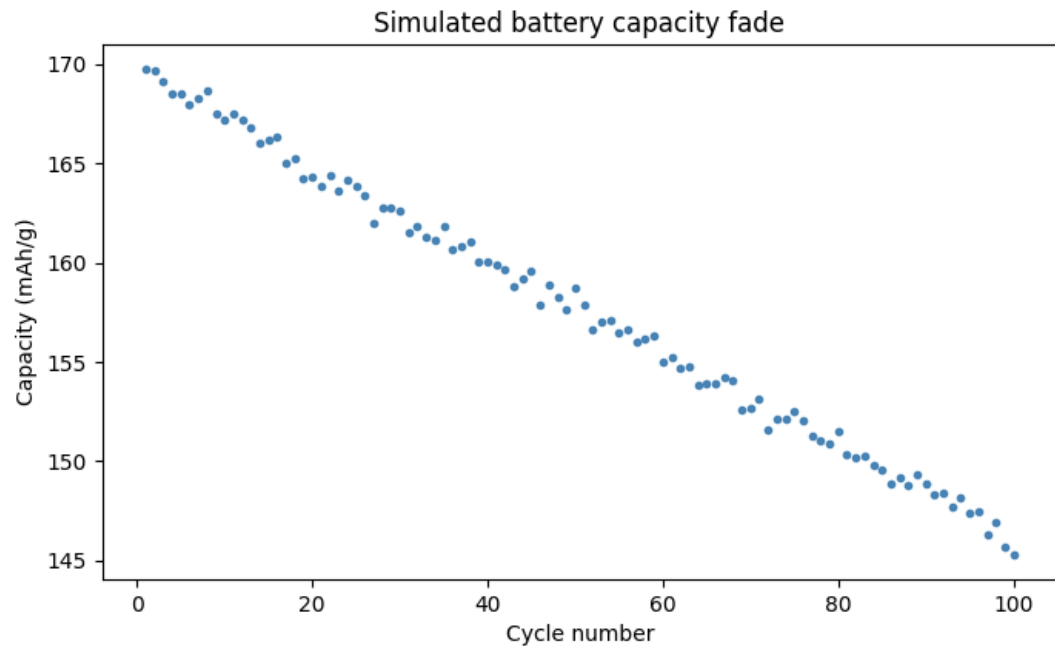
```
In [1]: import numpy as np
import matplotlib.pyplot as plt

rng = np.random.default_rng(7)          # <- try changing this seed and re-running!
cycle_numbers = np.arange(1, 101)
capacity_0 = 170.0                       # mAh/g
fade_rate = 0.0015                       # capacity loss per cycle

capacity = capacity_0 * np.exp(-fade_rate * cycle_numbers) + rng.normal(0, 0.5, 100)

plt.figure(figsize=(8, 4.5))
plt.plot(cycle_numbers, capacity, '.', color='steelblue')
plt.xlabel('Cycle number'); plt.ylabel('Capacity (mAh/g)')
plt.title('Simulated battery capacity fade')
plt.show()

print(f'Cycle 1:   {capacity[0]:.2f} mAh/g')
print(f'Cycle 100: {capacity[99]:.2f} mAh/g')
```



Cycle 1: 169.75 mAh/g

Cycle 100: 145.30 mAh/g

Try it live: change `rng = np.random.default_rng(7)` to a different number and re-run — same code, different noise, *same underlying truth*. This one cell already combines three Part I tools: **NumPy** (generates/computes the array), **pandas** (would label it), **Matplotlib** (renders it).

Collaboration: Git & GitHub

The same discipline as FAIR data principles (coming up next) — applied to code instead of data

The Problem: `analysis_v2_final REALLY.ipynb`

Renaming files by hand fails, and gets worse the longer a project runs:

- No record of **why** something changed
- No way to see exactly what changed between two versions without eyeballing both
- No safe way for two people to edit the same file at once without overwriting each other

Version control (git) fixes all three at once — every commit records exactly what changed, who, and why; the complete history is always available; two people's independent changes can usually be combined automatically.

A Commit Is Just a Labelled Snapshot

A tiny in-memory stand-in for git's commit history — the same mental model as the real thing.

```
In [2]: import hashlib
        from datetime import datetime, timedelta

        class ToyRepo:
            '''A tiny, in-memory stand-in for git's commit history.'''
            def __init__(self):
                self.commits = []          # chain of snapshots, oldest first
                self.working_files = {}     # current file contents

            def _commit_id(self, message, timestamp):
                raw = f'{message}{timestamp}{len(self.commits)}'.encode()
                return hashlib.sha1(raw).hexdigest()[:7]

            def commit(self, message, author='you'):
                parent = self.commits[-1]['id'] if self.commits else None
                timestamp = datetime.now() + timedelta(seconds=len(self.commits))
                snapshot = {'id': self._commit_id(message, timestamp), 'parent': parent,
                           'message': message, 'author': author, 'timestamp': timestamp,
                           'files': dict(self.working_files)}
                self.commits.append(snapshot)
                return snapshot['id']

            def log(self):
                for c in reversed(self.commits):
                    print(f"{c['id']} {c['timestamp']:%Y-%m-%d %H:%M:%S} {c['author']:<8} {c['message']}")

        repo = ToyRepo()
        repo.working_files['analysis.py'] = "print('load data')"
        repo.commit('Initial analysis script')

        repo.working_files['analysis.py'] += "\nprint('clean data')"
        repo.commit('Add data cleaning step')
```

```
repo.log()
```

```
65b3508 2026-08-27 12:58:43 you      Add data cleaning step
cf61b68 2026-08-27 12:58:42 you      Initial analysis script
```

Nothing is ever overwritten — an earlier snapshot is always reachable through the chain. Real commands: `git init` · `git add` · `git commit -m "message"` · `git log`.

Branches: Try an Idea Without Breaking What Works

Keep a stable line of commits untouched (`main`) while trying something risky on a separate line.

```
In [3]: class BranchingRepo(ToyRepo):
    def __init__(self):
        super().__init__()
        self.branches = {'main': None}    # branch name -> commit id it points to
        self.current_branch = 'main'

    def commit(self, message, author='you'):
        commit_id = super().commit(message, author)
        self.branches[self.current_branch] = commit_id
        return commit_id

    def branch(self, name):
        self.branches[name] = self.branches[self.current_branch]

    def checkout(self, name):
        self.current_branch = name

    def status(self):
        print(f"On branch '{self.current_branch}'")
        for name, cid in self.branches.items():
            marker = ' <-- you are here' if name == self.current_branch else ''
            print(f"  {name}: {cid}{marker}")

brepo = BranchingRepo()
brepo.working_files['model.py'] = "model = 'MLR'"
brepo.commit('Baseline: linear regression model')

brepo.branch('try-random-forest')    # fork off from the current commit
brepo.checkout('try-random-forest')
brepo.working_files['model.py'] = "model = 'RandomForest'"
brepo.commit('Experiment: swap in a Random Forest')
```

```
brepo.status()
```

On branch 'try-random-forest'

main: a561cfa

try-random-forest: 2324ca4 <-- you are here

main still points at the safe baseline commit — the experiment cannot break it, no matter how badly it goes. Same discipline as testing a risky package upgrade in a throwaway conda environment first (Notebook 6).

GitHub: Remotes and Working With Others

The vocabulary

- **clone** — download a full copy of a repository, history and all
- **push** — send your new local commits up to GitHub
- **pull** — download and merge commits someone else pushed

Two collaboration shapes

- **Direct collaborator**: added to a repo, branch → push → open a pull request for review
- **Fork**: no write access — make your own copy, change it, PR back to the original
- **Merge conflict**: two branches changed the same lines differently — git stops and asks you to decide by hand:

```
<<<<<< HEAD
model = 'RandomForest'
=====
model = 'GradientBoosting'
>>>>>> try-random-forest
```

Two Habits Worth Adopting From Day One

.gitignore — a plain-text list of patterns git should never track: build artefacts, large data files, anything that regenerates automatically. Large raw data files belong in a proper database (coming up), not committed directly — git keeps *every* version of every tracked file forever.

Notebooks in git — **.ipynb** files store cell *outputs* alongside the code, so re-running and re-saving changes the file even when no code changed. Fix: clear outputs before committing, or use **nbstripout** to do it automatically.

Worth setting up before your course-project notebook's history turns into hundreds of "re-ran cell 47" commits.

Break

5–10 minutes

Coming up after the break: **Part II — Data Handling & Materials Ontologies**, directly preparing both upcoming live-tutorial workshops.

Part II — Data Handling & Materials Ontologies

A composition might live in a colleague's spreadsheet, a synthesis temperature in a lab notebook, an XRD pattern in an instrument's proprietary format — each using slightly different names for the same thing.

This part is about closing that gap.

Why This Part Exists

Problem in the lab	Concept	Notebook
"I don't remember what column T2 meant"	Metadata	1
"Is 'tensile strength' the same as 'ultimate tensile stress'?"	Ontology (EMMO)	2
"I need my group's data queryable, not just files"	Database	3
"Where do I put data so others can find it?"	Repository (NOMAD)	4
"How do I apply this to MY dataset?"	Schema design	5

Two Live Workshops, Directly Ahead

Live Tutorial 1 — SQL & NoSQL in Practice

- Entirely local — no network, no account needed
- A realistic lab-scale investigation using Notebook 3's concepts

Live Tutorial 2 — The Real NOMAD API

- Connects to the real, public NOMAD repository
- No registration needed for the reading/plotting done there

Everything from here on is preparation for both.

Data Without Metadata Is (Almost) Useless

182, 195, 188, 201, 179, 193

What is this? Hardness values? In what units — Vickers, Brinell, Rockwell? Which sample? Which instrument? Measured when, by whom?

A bare number is scientifically worthless without its context

Metadata = data about data — exactly the context a lab notebook records almost unconsciously, and exactly what gets lost when data moves into a bare spreadsheet or plot.

The FAIR Principles

Principle		The plain-language question
F	Findable	Can a person or machine locate this dataset at all?
A	Accessible	Can it actually be fetched — even under conditions?
I	Interoperable	Can it combine with other data/tools with no manual translation?
R	Reusable	Is there enough context to reuse it correctly?

FAIR is a checklist, not a synonym for "open." Data can be FAIR and still access-controlled or paywalled.

Metadata in Python: Dictionaries and JSON

A dictionary is the natural structure for metadata — it's inherently nested. JSON is the standard interchange format (human-readable, near-lossless, what NOMAD uses under the hood).

```
In [4]: import json

cathode_record = {
    'identifier': 'urn:uuid:1234',
    'title': 'LiFePO4 cathode, solid-state synthesis batch 07',
    'licence': 'CC-BY-4.0',
    'creator': 'A. Lindqvist',
    'created': '2025-01-15',
    'method': 'solid-state reaction, 2-step calcination',
    'data': {'temperature_C': 750, 'time_h': 6, 'capacity_mAh_g': 161.4},
}

def fair_check(record):
    '''Which required FAIR-relevant fields, if any, are missing?'''
    required = ['identifier', 'title', 'licence', 'creator', 'created', 'method']
    missing = [k for k in required if k not in record]
    return not missing, missing

ok, missing = fair_check(cathode_record)
print('FAIR check passed:', ok)

# JSON round-trip: save, then load back
record_json = json.dumps(cathode_record, indent=2)
reloaded = json.loads(record_json)
print('Round-trip identical:', reloaded == cathode_record)
print(record_json)
```

```
FAIR check passed: True
Round-trip identical: True
{
  "identifier": "urn:uuid:1234",
  "title": "LiFePO4 cathode, solid-state synthesis batch 07",
  "licence": "CC-BY-4.0",
  "creator": "A. Lindqvist",
  "created": "2025-01-15",
  "method": "solid-state reaction, 2-step calcination",
  "data": {
    "temperature_C": 750,
    "time_h": 6,
    "capacity_mAh_g": 161.4
  }
}
```

This exact record reappears across the rest of today — rebuilt as an ontology individual, SQL rows, MongoDB documents, and a NOMAD-style archive.

The Problem: Words Are Not Reliable Identifiers

Synonymy — same concept, different words

- "yield strength" / "0.2% proof stress" / σ_y — a naive string filter misses the aliases

Homonymy — same word, different concepts

- "annealing" in metallurgy vs. molecular biology
- "conductivity" without a qualifier — electrical, or thermal?

A column header is just a string — it cannot disambiguate on its own.

Controlled Vocabulary → Taxonomy → Ontology

Level	What it provides	Example	Limitation
Controlled vocabulary	A flat, fixed term list	{as-cast, annealed, quenched}	No structure
Taxonomy	Vocabulary + an is-a hierarchy	Linnaean classification	One relation type only
Ontology	Taxonomy + relations, properties, formal logic	EMMO	Real upfront modelling effort

EMMO — The European Materials & Modelling Ontology

A **foundational, top-level ontology** for physical science and engineering.

- Developed/maintained by the European Materials Modelling Council (EMMC)
- Open, versioned, at a persistent ID (w3id.org/emmo), CC-BY-4.0
- Every concept ultimately derives from one root class: **Item**
- Domain ontologies build on top of it — e.g. **BattINFO** for FAIR battery data
- NOMAD increasingly annotates its own schemas with EMMO concepts

The Key Insight: A Measurement Is Not the Property

A material — Crystal Crystal → CrystallineMaterial → Material → OrdinaryMatter → PhysicalObject → ... → Item

A quantity — SpecificHeatCapacity SpecificHeatCapacity → Sign → SemioticEntity → Semiotics → ... → Item

(Real EMMO ancestor chains, walked live via SPARQL against the actual ontology.)

"188 HV" is a **Sign** representing a real hardness, not the hardness itself.

Try It: A Toy Ontology, Live

Building the same idea in miniature with `owlready2` — the same building blocks EMMO itself is built from.

```
In [5]: from owlready2 import get_ontology, Thing, ObjectProperty

onto = get_ontology("http://example.org/toy_emmo.owl")

with onto:
    class Item(Thing): pass
    class PhysicalEntity(Item): pass
    class Quantity(Item): pass
    class MaterialObject(PhysicalEntity): pass
    class CathodeMaterial(MaterialObject): pass
    class ThermodynamicTemperature(Quantity): pass

    class hasQuantitativeProperty(ObjectProperty):
        domain = [MaterialObject]
        range = [Quantity]

batch_07 = CathodeMaterial("LiFeP04_batch_07")
T_synthesis = ThermodynamicTemperature("T_synthesis_batch_07")
batch_07.hasQuantitativeProperty.append(T_synthesis)

print("Is batch_07 a MaterialObject?", isinstance(batch_07, MaterialObject))
print("Ancestors of CathodeMaterial:", [c.name for c in CathodeMaterial.ancestors()])
```

Is batch_07 a MaterialObject? True

Ancestors of CathodeMaterial: ['MaterialObject', 'Item', 'Thing', 'CathodeMaterial', 'PhysicalEntity']

Beyond the Hierarchy: What Makes an Ontology *Do* Things

A class hierarchy alone is just an organised list. The full Notebook 2 goes further, with four live demonstrations of what that structure actually buys you:

- **See it as a graph** — the same toy ontology drawn as an actual knowledge graph (`networkx`): classes and individuals as nodes, relations as labelled edges
- **Reasoning** — a real OWL-RL reasoner (`owlrl`) derives a fact that was *never explicitly written down*, purely from the axioms already in place
- **Closing the loop on synonymy** — two records using different labels ("yield strength" vs. "0.2% proof stress") are still found together by one class-based query, exactly the Section 2.1 problem, now solved
- **Catching a wrong unit** — the ontology flags a hardness value mistakenly recorded in a pressure unit, something a bare CSV column never would

This is the concrete answer to "why not just use a spreadsheet with good column names?"

Why a Database at All?

A dict / JSON file (previous section) works for a handful of records. It breaks down once you have:

- Thousands of records
- Multiple people writing at the same time
- Cross-record questions — filter, join, aggregate

Notebooks 1–2 solved a **meaning** problem. This solves an **infrastructure** problem.

Two Data Models: Rows vs. Documents

	Relational (SQL)	Document (NoSQL)
Shape	Fixed-column tables of rows	Nested, JSON-like documents
Schema	Enforced upfront	Flexible, per-document
Relationships	Foreign keys + JOIN	Usually embedded
Query language	SQL	Method-call API
Systems	PostgreSQL, MySQL, SQLite	MongoDB, CouchDB
Good fit for	Strictly tabular, strict integrity	Heterogeneous, nested data

SQL, Live

A small relational schema — `samples` and `results`, linked by a foreign key. Watch what happens when a `NOT NULL` constraint is violated.

```
In [6]: import sqlite3
import pandas as pd

conn = sqlite3.connect(':memory:')
conn.execute('''CREATE TABLE samples (
    sample_id TEXT PRIMARY KEY, material TEXT, method TEXT NOT NULL
)''')
conn.execute('''CREATE TABLE results (
    sample_id TEXT PRIMARY KEY, capacity_mAh_g REAL,
    FOREIGN KEY(sample_id) REFERENCES samples(sample_id)
)''')

samples = [('batch_05', 'LiFePO4', 'solid-state'), ('batch_06', 'LiFePO4', 'sol-gel'),
            ('batch_07', 'LiFePO4', 'solid-state')]
conn.executemany('INSERT INTO samples VALUES (?, ?, ?)', samples)
results = [('batch_05', 158.2), ('batch_06', 163.9), ('batch_07', 161.4)]
conn.executemany('INSERT INTO results VALUES (?, ?)', results)
conn.commit()

df = pd.read_sql_query('''
    SELECT s.sample_id, s.method, r.capacity_mAh_g
    FROM samples s JOIN results r ON s.sample_id = r.sample_id
''', conn)
print(df)

# Schema enforcement, Live: try to insert a sample with no method
try:
    conn.execute("INSERT INTO samples VALUES ('batch_08', 'LiFePO4', NULL)")
except sqlite3.IntegrityError as e:
    print('\nRejected:', e)
```

	sample_id	method	capacity_mAh_g
0	batch_05	solid-state	158.2
1	batch_06	sol-gel	163.9
2	batch_07	solid-state	161.4

Rejected: NOT NULL constraint failed: samples.method

Document Store, Live

The same 5-batch scenario as MongoDB-style documents (via `mongomock`, no server needed). Watch what a *missing* field does — nothing.

```
In [7]: import mongomock

client = mongomock.MongoClient()
db = client.cathode_db
db.batches.insert_many([
    {'sample_id': 'batch_05', 'method': 'solid-state',
     'steps': [{'name': 'mixing'}, {'name': 'calcination'}],
     'results': {'capacity_mAh_g': 158.2}},
    {'sample_id': 'batch_06', 'method': 'sol-gel',
     'steps': [{'name': 'gelation'}],
     'results': {'capacity_mAh_g': 163.9}},
])
# A document missing 'results' entirely -- accepted without error, no migration needed
db.batches.insert_one({'sample_id': 'batch_10', 'method': 'solid-state', 'steps': []})

for doc in db.batches.find({'method': 'sol-gel'}):
    print(doc['sample_id'], doc['results'])

print("\nbatch_10 has a 'results' field?", 'results' in db.batches.find_one({'sample_id': 'batch_10'}))
```

```
batch_06 {'capacity_mAh_g': 163.9}
```

```
batch_10 has a 'results' field? False
```

"Document databases trade upfront enforcement for upfront flexibility — which one you want depends on how settled your data's shape already is."

Choosing a Model

	Relational (SQL)	Document (NoSQL)	Domain repository (NOMAD)
Schema enforcement	Strict, built-in	Flexible / opt-in	Strict, validated
Best for	Fixed-shape registries	Heterogeneous exports	FAIR, ontology-aware data
You set up yourself	Yes	Yes	No — used as infrastructure
Ontology-aware	No	No	Yes — EMMO concept tags

► Coming Up: Live Tutorial 1 — SQL & NoSQL in Practice

- Grow the dataset: 5 batches → 45 batches across 3 calcination furnaces
- Investigate **"the furnace mystery"** — is Furnace-C really under-performing?
- Answer the same question independently in SQL and MongoDB, then cross-check
- See schema enforcement differences live
- Optional: indexing and query performance at real scale (200,000 rows)

Entirely local. No accounts, no installs, no internet needed.

What Is NOMAD?

An **open-source, FAIR data infrastructure** for materials science (nomad-lab.eu).

- Developed by **FAIRmat** — part of Germany's National Research Data Infrastructure (NFDI)
- Started for computational/DFT/simulation data
- Has grown **Electronic Lab Notebook (ELN)** support for experimental data

NOMAD Oasis: the same software, run as your own private instance — for unpublished results or embargoed data, speaking the same data model as public NOMAD.

Anatomy of a NOMAD Entry

Layer	What it is	Analogy from earlier today
Raw data	The original files, untouched	The instrument's raw export
Archive	Parsed into NOMAD's structured schema	The FAIR <code>cathode_record</code> dict — now machine-validated
Metadata	Searchable summary fields	The F / A / R fields from <code>fair_check</code>

Where EMMO Fits In, Live

NOMAD schemas attach an ontology **concept IRI** to individual quantities, so differently-named fields in different labs can still be recognised as the same concept.

```
In [8]: nomad_style_entry = {
    'metadata': {'upload_id': 'upload_2025_01_cathodes', 'entry_id': 'entry_LiFePO4_batch07',
                 'authors': ['A. Lindqvist'], 'licence': 'CC-BY-4.0'},
    'archive': {'data': {
        'm_def': 'CathodeSynthesisEntry',
        'synthesis': {'method': 'solid-state reaction'},
        'results': {'capacity_mAh_g': 161.4,
                    'quantity_kind_iri': 'emmo:ElectricCharge'},    # <- the EMMO tag
    }},
}

def validate_entry(entry, schema):
    '''A minimal structural validator -- same spirit as fair_check, checking
    structure instead of FAIR-ness.'''
    for key, expected_type in schema.items():
        node = entry
        for part in key.split('.'):
            if part not in node:
                return False, f'missing {key}'
            node = node[part]
        if not isinstance(node, expected_type):
            return False, f'{key} has wrong type'
    return True, None

schema = {
    'archive.data.synthesis.method': str,
    'archive.data.results.capacity_mAh_g': (int, float),
}
print(validate_entry(nomad_style_entry, schema))
```

(True, None)

► Coming Up: Live Tutorial 2 — The Real NOMAD API

- Query the real, public, live NOMAD repository over the internet — 759,000+ iron-containing entries
- A search-semantics gotcha: "all" these elements vs. "exactly" these elements
- Paginate through hundreds of results with a cursor, not page numbers
- Two-step pattern: search the index (fast) → fetch full archives (selective)
- Unit discipline: NOMAD stores SI units — convert J → eV before anything is meaningful
- Case study: computed (DFT) band gaps for ZnO systematically under-estimate the real value
- Optional: a personal access token — what changes once you're authenticated

No account needed for ~90% of the session — stable internet required.

Notebook 5 — Putting It All Together

Design schema → **Builder function** → **Generate** (12 samples) → **Validate** (every record) → **Export** (CSV + JSON archive) → **Round-trip check** → **FAIR-check** (dataset level)

A 12-sample tool-steel hardness study (temperature × time × quench medium) — the exact shape of deliverable you'll produce for the real course project.

*Honest gap: hardness has no concept in core EMMO — the record uses a clearly-marked local placeholder (**Local:VickersHardness**) instead of inventing a fake one.*

This Is Your Course Project, in Miniature

- A schema that separates what you controlled from what you measured
- A validated, EMMO-tagged record for every sample — not just some
- Both a clean analysis-ready table **and** a full metadata archive

Forward link: the exported CSV is exactly the file format Part III's statistics notebooks expect — group means, ANOVA across temperature and quench medium.

Before the Workshops: Checklist

A working Python environment

- Easiest: click **Open in Colab** on any book page — zero install, runs in your browser
- Or local (Anaconda/Miniconda) + `requirements.txt`, if you prefer your own machine

Live Tutorial 1 (SQL & NoSQL)

- Nothing extra — entirely local, no accounts or servers

Live Tutorial 2 (NOMAD API)

- Stable internet access
- Optional: a free NOMAD account + Personal Access Token

Key Takeaways From Today

1. Simulated data with a known true answer lets us check whether a method actually works
2. Git and GitHub make analysis reproducible and collaborative
3. FAIR + EMMO ensure data means the same thing to different people and software
4. SQL vs. NoSQL is a genuine trade-off — enforcement vs. flexibility
5. NOMAD is a concrete, worked FAIR repository — two workshops now put this in your hands

Coming Up

Next: Live Tutorial 1 — SQL & NoSQL in Practice [chapters/02_data_handling/06_live_tutorial_1_sql_nosql.ipynb](#)

Then: Live Tutorial 2 — The Real NOMAD API [chapters/02_data_handling/07_nomad_api_live_tutorial.ipynb](#)

Later: Part III — Applied Statistics

Questions?

1KB570 — Data Handling, Statistics & Design of Experiments for Materials Scientists

Department of Chemistry – Ångström, Uppsala University